- **性能分析**
  - 使用工具：performance
  - 说明
    - 查看是否有过长的任务
    - 定位代码

- **优化：复杂js执行**
  - 另起线程
    - 参考：Web Workers
    - 示例代码

```
// 主线程
const worker = new Worker(new URL('./worker.ts', import.meta.url).href) // 基于vite环境的引用路径
worker.postMessage('') // 执行开始
worker.addEventListener('message', (event: MessageEvent<any>) => {
  // 执行结束
})

// worker.ts
self.addEventListener('message', (event: MessageEvent<any>) => {
  // 执行复杂代码
  self.postMessage('') // 传输执行结果
})
```

    - 说明
      - 使用window.postMessage与主线程通信
        - 参数受到结构化克隆算法限制
      - worker内无法访问window（可用self代替）
      - worker内无法操作DOM
  - 函数切片
    - 参考： 生成器函数
    - 示例代码

```
// 复杂代码
function* complexCalculate() {
  let count = 0
  const start = performance.now()
```

```
  while (performance.now() - start <= 5000) {
    yield count++
  }
  return count
}

// 将复杂代码切片为异步函数
function timeSlice(): Promise<number> {
  return new Promise(resolve => {
    const iteratorFn = complexCalculate()
    const recursion = () => {
      const start = performance.now()
      let result: IteratorResult<unknown, number>
      do {
        result = iteratorFn.next()
      } while (!result.done && performance.now() - start < 16) // 切片为16ms长度的任务
      if (result.done) {
        resolve(result.value)
      } else {
        requestAnimationFrame(recursion)
      }
    }
    recursion()
  })
}
```

    - 说明
      - 将原本的长任务切片成多个短任务
      - 会降低执行效率，切片越小效率越低

- **其他性能优化点**
  - 避免不必要的响应式数据转换
    - 数据整体无需响应
      - 不使用响应式api，直接声明
    - 数据部分无需响应
      - markRaw()
      - shallowRef()
      - shallowReactive()
  - 使用css或requestAnimationFrame绘制动画

- 中断已经不需要了的请求
  - AbortController
- 减少排列、绘制：使用transform实现移动效果、合并dom操作
- 避免强制排列
  - 避免大型、复杂的布局和布局抖动
- 页面缓存后关闭侦听器
- 按需加载、页面缓存
- 高频操作节流处理，例如滚动、移动事件
- 让传给子组件的 props 尽量保持稳定，参考Props 稳定性
- 使用v-once、v-memo